

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

Écriture d'algorithmes séquentiels et structures de contrôle

Module 1 : Algorithmique

Consigne : Les algorithmes sont écrits en pseudo-code normalisé (variables typées, structures de contrôle indentées). Justifiez chaque choix structurel.

EXERCICE 1 Gestion de stock et seuils d'alerte

Une entreprise gère un stock d'articles. Pour chaque article, on connaît : le code (chaîne de 5 caractères), la désignation (chaîne), le prix unitaire HT (réel), la quantité en stock (entier), le seuil de réapprovisionnement (entier) et le taux de TVA applicable (réel, ex : 0.1925). On souhaite un algorithme traitant une liste d'articles (taille maximale $MAX=100$) pour éditer un état des lieux.

1. Déclarations : Proposez une structure de données adaptée (enregistrement/tableau) pour stocker les informations d'un article. Déclarez les variables nécessaires au traitement principal, y compris les compteurs et accumulateurs pour les totaux. (2 pts)
2. Saisie contrôlée : Écrivez l'algorithme de saisie d'un article. Le code doit être unique (vérification dans le tableau), le prix et les quantités strictement positifs, le taux TVA dans $\{0, 0.055, 0.10, 0.1925\}$. Utilisez une structure Répéter... Jusqu'à pour chaque champ invalide. (3 pts)
3. Traitement itératif : Une fois le stock saisi (fin de saisie par code "FIN"), écrire la boucle principale (TantQue ou Pour) qui parcourt le stock effectif pour :
 - Calculer la valeur HT et TTC du stock pour l'article.
 - Déterminer l'état : "Rupture" ($qté=0$), "Alerte" ($0 < qté \leq \text{seuil}$), "Normal".
 - Afficher une ligne formatée : Code | Désignation | Qté | État | Valeur HT | Valeur TTC.
 - Accumuler la valeur totale HT et TTC du stock global.(3 pts)
4. Analyse critique : L'algorithme utilise un tableau de taille fixe MAX . Proposez une modification structurelle (sans changer de langage) pour gérer un nombre illimité d'articles, en décrivant l'impact sur les boucles de parcours et la mémoire. (2 pts)

EXERCICE 2 Calcul d'impôt progressif par tranches

L'impôt sur le revenu (IR) est calculé par tranches sur le revenu net imposable (RNI) selon le barème annuel suivant (montifs en FCFA) :

Tranche de RNI	Taux	Imposition de la tranche
[0; 2 000 000]	0%	0
]2 000 000; 5 000 000]	10%	$(\text{RNI} - 2\,000\,000) \times 0.10$
]5 000 000; 10 000 000]	15%	$300\,000 + (\text{RNI} - 5\,000\,000) \times 0.15$
> 10 000 000	25%	$1\,050\,000 + (\text{RNI} - 10\,000\,000) \times 0.25$

Un abattement de 20% est appliqué sur le revenu brut (RB) pour obtenir le RNI, plafonné à 3 000 000 FCFA. L'IR final ne peut être inférieur à un minimum de perception de 5 000 FCFA si $\text{RNI} > 0$.

1. Variables et constantes : Déclarez toutes les constantes symboliques du barème (bornes, taux, cumulatifs) et les variables de travail. Justifiez le type Réel vs Entier. (2 pts)
2. Structure alternative complète : Écrivez l'algorithme calculant l'IR à partir du RB saisi. Utilisez une structure Si...SinonSi...Sinon imbriquée ou séquentielle pour le barème. Gérez l'abattement plafonné et le minimum de perception. (3 pts)
3. Version itérative (Généralisation) : On souhaite rendre l'algorithme paramétrable : les bornes, taux et cumulatifs sont stockés dans trois tableaux BORNE[1..N], TAUX[1..N], CUMUL[1..N] (dernière borne = INFINI). Réécrivez le calcul de l'IR avec une boucle Pour i de 1 à N Faire unique. Expliquez comment la condition d'arrêt naturelle de la boucle remplace les Si successifs. (3 pts)
4. Test de non-régression : Donnez 4 valeurs de RB (dont une limite de tranche, une valeur nulle, une valeur très élevée) et le résultat attendu pour valider l'algorithme. (2 pts)

EXERCICE 3 Analyse et correction d'algorithmes itératifs

On considère les trois algorithmes ci-dessous censés calculer la somme $S = \sum_{k=1}^n \frac{1}{k^2}$ pour un entier $n \geq 1$ saisi.

```
// Algo A
S <- 0
Pour k de 1 à n Faire
    S <- S + 1 / (k * k)
FinPour
Afficher S
\end{verbatim}

\begin{verbatim}
// Algo B
S <- 0 ; k <- 1
TantQue k <= n Faire
```

```

    S <- S + 1 / (k * k)
    k <- k + 1
FinTantQue
Afficher S

// Algo C
S <- 0 ; k <- 0
Repeter
    k <- k + 1
    S <- S + 1 / (k * k)
JusquA k = n
Afficher S

```

1. **Équivalence** : Prouvez par récurrence (ou invariant de boucle) que les trois algorithmes produisent le même résultat pour tout $n \geq 1$. Identifiez l'invariant de boucle commun. (3 pts)
2. **Cas limite et robustesse** : Que se passe-t-il pour chaque algorithme si l'utilisateur saisit $n = 0$? $n = -5$? Proposez une modification unique (garde-fou) à placer avant la boucle pour garantir un comportement correct (affichage 0 ou message d'erreur) sans changer le corps des boucles. (2 pts)
3. **Précision et ordre d'évaluation** : En arithmétique flottante (simple précision), l'ordre de la sommation influence le résultat. Proposez une version modifiée de l'Algo A (boucle `Pour`) qui somme les termes du plus petit au plus grand ($k = n$ downto 1). Écrivez l'en-tête de boucle exact. Justifiez pourquoi cela améliore la précision numérique. (2 pts)
4. **Complexité** : Donnez la complexité en temps et en espace de ces algorithmes. Est-il possible de calculer S en $O(1)$ espace et $O(\log n)$ temps ? Justifiez. (3 pts)

EXERCICE 4 Simulation de système à événements discrets : File d'attente

On modélise une file d'attente M/M/1/10 (capacité 10 clients max, 1 serveur). L'horloge simulation H avance par pas variable. Variables d'état : `File` (entier, nb clients en attente), `Serveur` (booléen : Vrai = occupé), `H_Proch_Arrivee`, `H_Proch_Depart` (réels). Constantes : λ (taux arrivée), μ (taux service). Fonction `AleaExp( $\lambda$ )` renvoie un aléa exponentiel.

1. **Initialisation** : Écrivez le bloc d'initialisation ($H = 0$, état vide, premier événement arrivée programmé). (1 pt)
2. **Boucle principale** : La simulation s'arrête à $H = T_{MAX}$. Écrivez la structure `TantQue  $H < T_{MAX}$  Faire` qui :
 - (a) Détermine l'événement imminent (arrivée ou départ) et avance H .

- (b) Traite l'arrivée : si $file < 10$ et serveur libre $\rightarrow$ service immédiat (prog. départ) ; si $file < 10$ et serveur occupé $\rightarrow file++$; si $file = 10 \rightarrow$ client perdu (compteur $Perdu++$).
Progr. prochaine arrivée.
- (c) Traite le départ : serveur libre ; si $file > 0 \rightarrow file-$, serveur occupé, prog. départ ; sinon serveur reste libre.
- (4 pts)
3. Statistiques : À la fin, on veut le taux de perte ($Perdu/Total_Arrives$) et le taux d'occupation du serveur (durée occupée / T_{MAX}). Quelles variables accumulateurs faut-il ajouter dans la boucle ? Comment calculer la durée occupée sans stocker l'historique ? (2 pts)
4. Analyse de la condition d'arrêt : La condition $H < T_{MAX}$ suffit-elle si le dernier événement programmé est une arrivée à $H = T_{MAX} - \epsilon$ qui génère un départ après T_{MAX} ? Proposez une condition d'arrêt robuste garantissant le traitement de tous les événements $\leq T_{MAX}$. (3 pts)